HEIG-VD / INGÉNIERIE DES MÉDIAS / PROGRAMMATION WEB

CLASSES

```
class Person {

}
```

Comme mentionné lors du cours précédent, les classes en JavaScript sont un “sucre syntaxique”.

En arrière plan, c’est bien le système de Prototypes qui est implémenté.

HEIG VD

```
class Person {
  constructor(name, year) {
    this.name = name;
    this.year = year;
  }
}
```

La méthode `constructor()` est l'équivalent d'une function constructor vue lors de du cours précédent.

Son rôle est de créer et initialiser un objet instance. On peut évidemment passer des paramètres à cette méthode.

Cette méthode est facultative, mais c'est à cet endroit que devrait être fait toute initialisation personnalisée.

HE IG VD

```
class Person {
  constructor(name, year) {
    this.name = name;
    this.year = year;
  }
}

const anna = new Person("Anna", 1985);
```

Comme précédemment, nous pouvons créer une nouvelle instance de Person avec le mot-clé `new`.

```
class Person {
  constructor(name, year) {
    this.name = name;
    this.year = year;
  }
  calcAge() {
    return 2023 - this.year;
  }
}

const anna = new Person("Anna", 1985);
anna.calcAge() // 38
```

Nous pouvons déclarer des méthodes dans le bloc de classe.

Attention, ces méthodes ne seront pas copiées sur chaque instance. Elles seront stockées dans `Person.prototype`, comme auparavant.

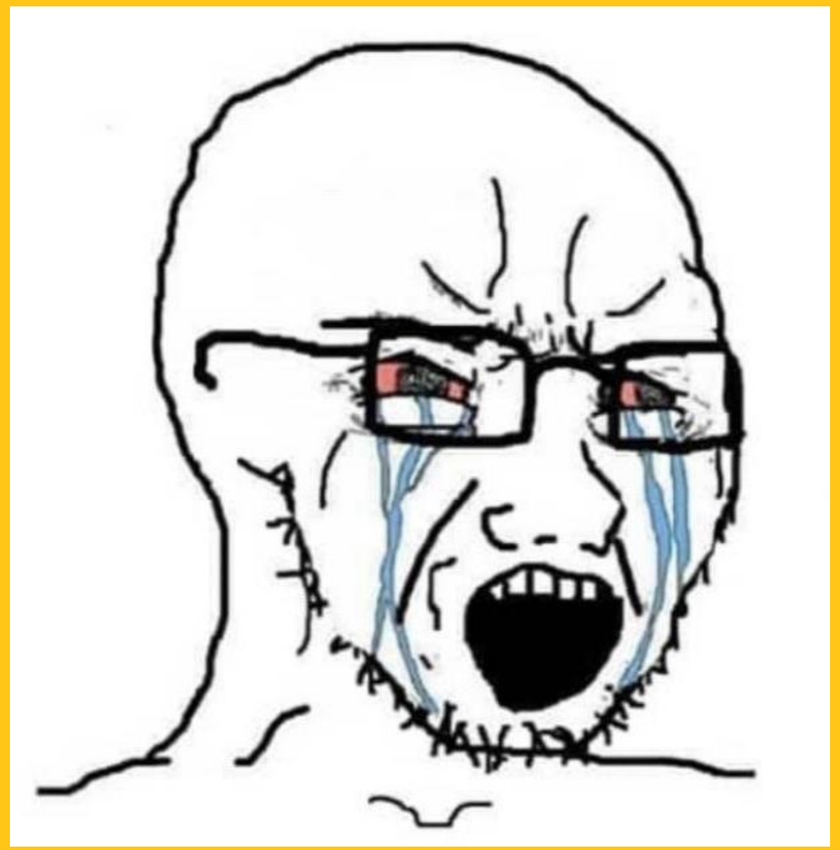
HEIG VD

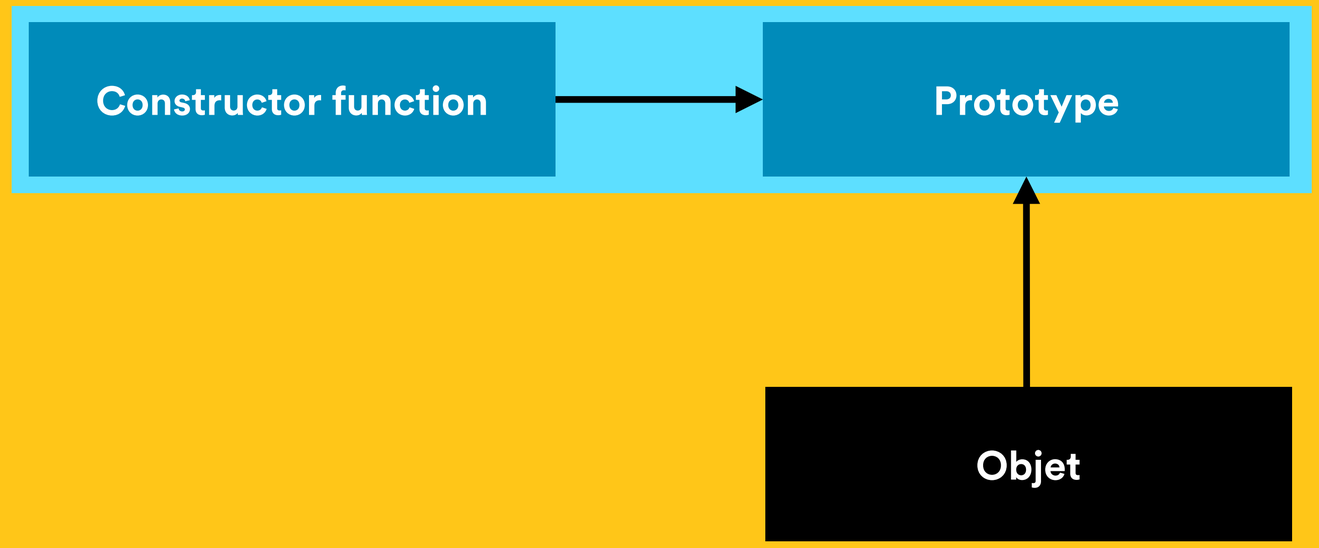
Voilà ce qu’on entend par “sucre syntaxique”.
Ça marche exactement comme avant, mais c’est plus agréable à écrire, raisonner et maintenir.

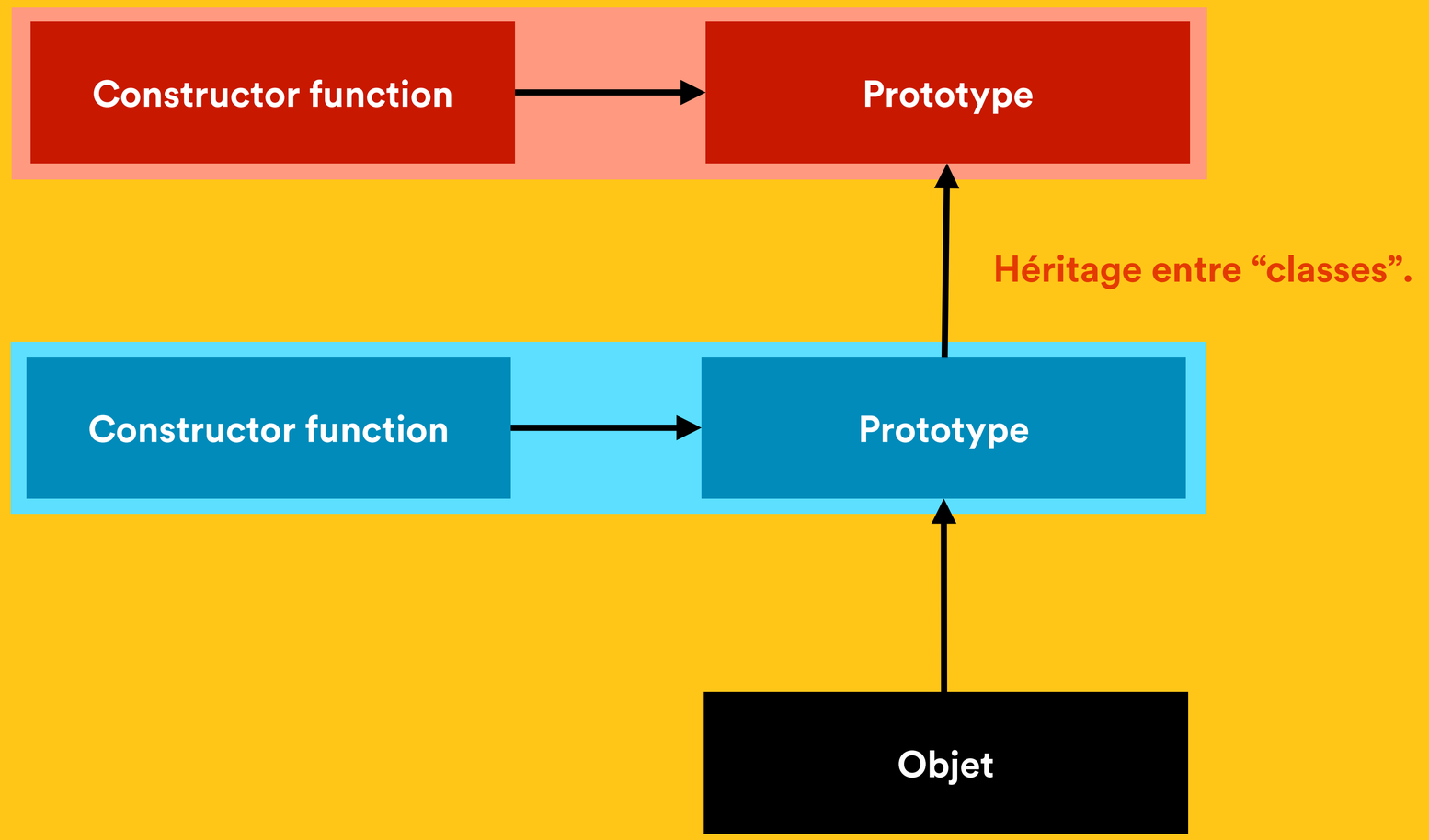
1. Les classes ne sont pas “hoisted”. Vous devez les déclarer avant de les utiliser.

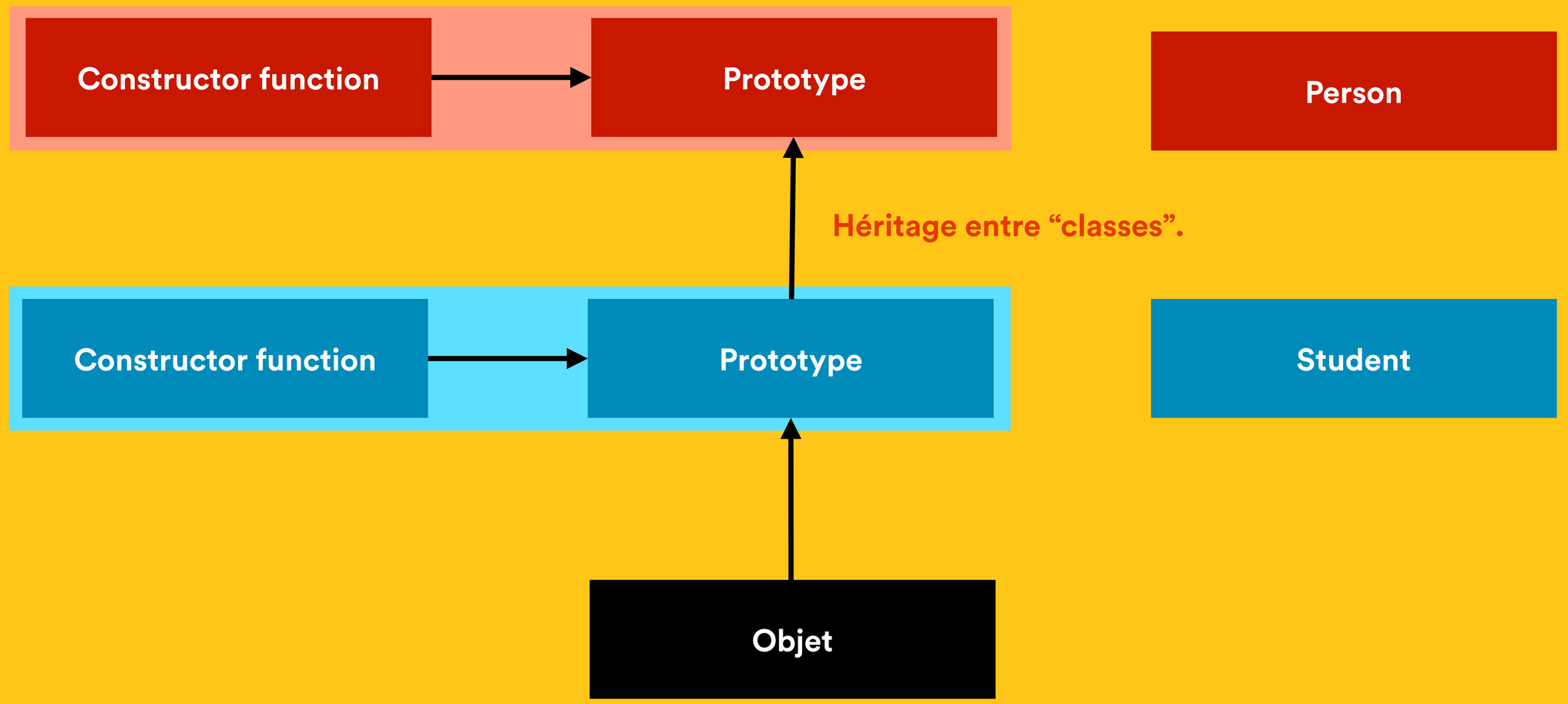
2. Les classes sont des citoyennes de première classe.

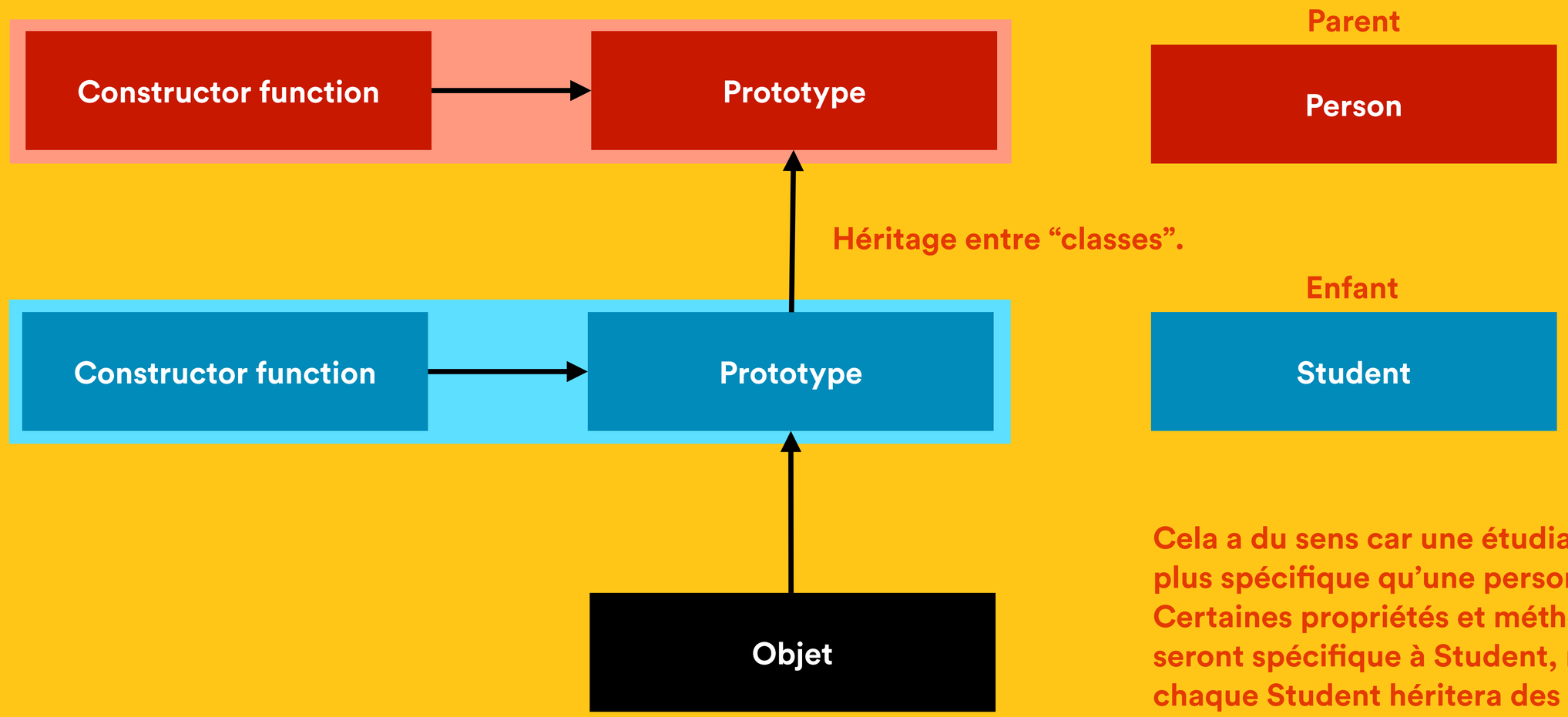
3. Les classes sont exécutée en strict mode.




Constructor function
Prototype
Objet


Constructor function
Prototype
Héritage entre “classes”.
Constructor function
Prototype
Objet


Constructor function
Prototype
Person
Héritage entre “classes”.
Constructor function
Prototype
Student
Objet



Cela a du sens car une étudiante est plus spécifique qu’une personne. Certaines propriétés et méthodes seront spécifique à Student, mais chaque Student héritera des propriétés et méthodes de Person.

HEIG VD

```
class Person {
…
}

class Student extends Person {
}
```

Le mot-clé extends qui peut suivre une déclaration de classe rend le lien entre Prototypes implicite.

```
class Student extends Person {
  constructor(name, year, major) {
    super(name, year);
    this.major = major;
  }
}
```

La fonction `super()` se réfère au `consctructor()` de la classe parent.

```
class Student extends Person {
}
```

Si la classe enfant n’a pas de propriétés supplémentaires, vous pouvez carrément omettre `constructor()` et `super()`

HEIG VD